МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №3

по дисциплине

«Технологии компонентного программирования»

«Brython: клиентский сайт на Python»

Выполнил:
Лебедевич Артем Владимирович
магистрант группы 556301

Преподаватель:
Герман Олег Витольдович
кандидат технических наук
доцент кафедры ИТАС

Минск, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы 2

2 Теоретические сведения 3
2.1 Brython 3
2.2 Различия версий 3
2.3 Корни многочлена 4
2.4 Линейная регрессия 4
2.5 Оптимум функции без ограничений 4

3 Реализация 6
3.1 Структура программы 6
3.2 Страница 6
3.3 Корни многочлена 6
3.4 Линейная регрессия 7
3.5 Оптимум 7
3.6 Окно и сервер 8

4 Результаты работы программы 9

5 Ответы на контрольные вопросы 13

Заключение 14

Список использованных источников 15

1 ЦЕЛЬ РАБОТЫ

Изучить возможности фреймворка Brython, позволяющего писать клиентские сценарии интернет-приложений на языке Python. Создать клиентский сайт, который находит корни многочлена, строит прогноз с помощью линейной регрессии и находит оптимальное значение функции без ограничений.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Brython

Браузер выполняет только JavaScript. Brython — это программа на JavaScript (`brython.js`), которая находит на странице блоки `<script type="text/python">`, переводит исходный текст на Python в JavaScript прямо в браузере и выполняет его. Благодаря пакету `browser` программа получает доступ к объектам DOM, поэтому обработчики событий, разметка и рисование на холсте пишутся на Python. Основные понятия приведены в таблице 2.1.

Таблица 2.1 – Основные понятия Brython

Понятие	Смысл
`brython.js`	компилятор и среда выполнения, подключаются тегом `<script src>`
`brython_stdlib.js`	стандартная библиотека Python в виде одного файла
`brython()`	функция, которая находит и запускает сценарии Python; вызывается из `<body onload>`
`browser.document`	документ DOM: `document["id"]` — элемент, `elt.bind("click f)` — обработчик события
`browser.html`	конструкторы тегов: `html.H2("текст")`; `document <= элемент` добавляет элемент на страницу
`pythonpath`	каталоги, где `import` ищет собственные модули (они загружаются запросом XHR)

Методические материалы рекомендуют не подключать Brython из облака, а использовать локальную установку. Команда `pip install brython` копирует файлы `brython.js` и `brython_stdlib.js` в каталог пакета; скрипт `scripts/fetch_runtimes.py` переносит их в каталог сайта.

2.2 Различия версий

Методичка написана для Brython 3.9, в работе использована версия 3.15.0. При переходе обнаружились три отличия, которые влияют на программу.

– Инструкция `import browser` больше не делает доступным `browser.html`: нужно писать `import browser.html` или `from browser import html`.
– Brython кэширует стандартную библиотеку в IndexedDB. Браузер без окна, который снимает DOM сразу после события `load`, видит недостроенную страницу; параметр `brython({indexedDB: false})` делает

запуск последовательным.

– Форматирование `"%g"% x` неверно для очень малых и очень больших чисел (`2.2e-16` превращается в `0e+00`), тогда как `"{:g}".format(x)` работает правильно. Поэтому сайт форматирует числа методом `format`.

2.3 Корни многочлена

Для многочлена $p(x) = a_0 x^n + \cdots + a_n$ все корни, в том числе комплексные, находятся методом Дурана — Кернера. После деления на старший коэффициент берётся n начальных приближений на окружности, и каждое пересчитывается по формуле

$$z_i \leftarrow z_i - \frac{p(z_i)}{\prod_{j \neq i}(z_i - z_j)}.$$

Итерации прекращаются, когда наибольшая поправка меньше 10^{-14}. Затем корни уточняются несколькими шагами Ньютона по исходному многочлену. Многочлены первой и второй степени решаются по формулам (для второй степени — в устойчивой форме, без вычитания близких чисел), нулевые последние коэффициенты дают точные нулевые корни. Малые мнимые части (порядка 10^{-9}) считаются вычислительным шумом и отбрасываются.

2.4 Линейная регрессия

По точкам (x_i, y_i) строится прямая $y = ax + b$ методом наименьших квадратов:

$$a = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \qquad b = \bar{y} - a\bar{x}.$$

Качество оценивается коэффициентом детерминации $R^2 = 1 - \sum e_i^2 / \sum (y_i - \bar{y})^2$ и среднеквадратичной ошибкой. Прогноз — значение прямой в следующих точках с шагом, равным среднему расстоянию между исходными x. Для каждой точки прогноза строится приближённая 95 % полоса $\pm 1{,}96\, s \sqrt{1 + 1/n + (x_0 - \bar{x})^2 / \sum (x_i - \bar{x})^2}$, где s — остаточное стандартное отклонение.

2.5 Оптимум функции без ограничений

Реализованы три метода.

– **Метод Нелдера — Мида** (симплекс) не требует производных: из $n + 1$ вершин заменяется худшая — отражением, растяжением или сжатием, при неудаче весь симплекс сжимается к лучшей вершине. Метод проходит узкую долину функции Розенброка.

– **Градиентный спуск** движется против градиента, который вычисляется

центральными разностями; шаг подбирается делением пополам, пока не выполнится условие Армихо $f(x - t\nabla f) \le f(x) - 10^{-4} t \|\nabla f\|^2$.

– **Золотое сечение** находит минимум унимодальной функции одной переменной на заданном отрезке; отрезок сужается в $(\sqrt{5} - 1)/2$ раз за шаг.

Максимум ищется как минимум функции $-f$. Функция вводится текстом (знак `^` означает степень), текст один раз компилируется в `lambda`, которой видны только имена из модуля `math`; любое другое имя вызывает ошибку.

3 РЕАЛИЗАЦИЯ

3.1 Структура программы

Листинг 3.1 – Структура лабораторной работы

```
lab3/numerics.py           численные методы, только math и cmath
lab3/site/index.html       страница: три вкладки, поля ввода, холсты
lab3/site/app.py           код страницы на Python (запускается Brython)
lab3/site/examples.html    примеры из методички
lab3/site/brython*.js      Brython 3.15.0 (копируется скриптом)
lab3/server.py             HTTP-сервер каталога lab3/
lab3/cases.py              проверяемые сценарии
lab3/runner.py             запуск сценариев из командной строки
lab3/gui.py                окно: страница в WebView и список сценариев
```

Главная особенность решения: файл `numerics.py` один. Сервер отдаёт его браузеру по адресу `/numerics.py`, страница подключает его инструкцией `import numerics` (параметр `pythonpath: ['/']`), а тесты в CPython импортируют тот же файл.

3.2 Страница

Разметка содержит заголовок, три кнопки вкладок и три раздела. Brython подключается в конце страницы, а запускается по событию загрузки (листинг 3.2).

Листинг 3.2 – Подключение Brython

```
<body onload="brython({pythonpath: ['/'], indexedDB: false})">
  ...
  <script src="brython.js"></script>
  <script src="brython_stdlib.js"></script>
  <script type="text/python" src="app.py"></script>
</body>
```

3.3 Корни многочлена

Обработчик кнопки читает коэффициенты, вызывает `poly_roots` и строит таблицу средствами `browser.html` (листинг 3.3); корни отмечаются точками на комплексной плоскости.

Листинг 3.3 – Обработчик вкладки корней

```
def run_roots(ev=None):
    box = document["roots_result"]
    try:
        coeffs = nm.poly_trim(nm.parse_numbers(document["roots_coeffs"].value))
        roots = nm.poly_roots(coeffs)
    except ValueError as exc:
        show_error(box, str(exc))
        return
    box.clear()
    box <= html.DIV("p(x) = " + nm.poly_text(coeffs), Class="big")
    rows = [[str(i + 1), nm.root_text(r), "{:.2e}".format(abs(nm.poly_eval(coeffs, r)))]
            for i, r in enumerate(roots)]
```

```
    box <= table(["#", "root", "|p(root)|"], rows)
```

Основная часть метода Дурана — Кернера (листинг 3.4) — обычный цикл по комплексным числам, встроенный тип `complex` поддерживается Brython.

Листинг 3.4 – Метод Дурана — Кернера

```
monic = [c / coeffs[0] for c in coeffs]
radius = _cauchy_bound(monic)
guess = [radius * 0.5 * cmath.exp(2j * math.pi * k / degree + 0.4j) for k in range(degree)]
for _ in range(max_iter):
    largest = 0.0
    for i in range(degree):
        denominator = 1
        for j in range(degree):
            if i != j:
                denominator *= guess[i] - guess[j]
        step = poly_eval(monic, guess[i]) / denominator
        guess[i] -= step
        largest = max(largest, abs(step))
    if largest < tol:
        break
```

3.4 Линейная регрессия

Наблюдения вводятся строками `x y`; строка с одним числом задаёт ряд $y(1), y(2), \ldots$ Функция `linear_regression` возвращает коэффициенты и показатели качества, `forecast` продолжает ряд (листинг 3.5).

Листинг 3.5 – Регрессия и прогноз

```
def linear_regression(xs, ys):
    n = len(xs)
    mean_x, mean_y = sum(xs) / n, sum(ys) / n
    sxx = sum((x - mean_x) ** 2 for x in xs)
    sxy = sum((x - mean_x) * (y - mean_y) for x, y in zip(xs, ys))
    slope = sxy / sxx
    intercept = mean_y - slope * mean_x
    ...

def forecast(model, xs, steps):
    step = (xs[-1] - xs[0]) / (len(xs) - 1) if len(xs) > 1 else 1.0
    future = [xs[-1] + step * (k + 1) for k in range(int(steps))]
    return [(x, predict(model, x)) for x in future]
```

3.5 Оптимум

Функция из текстового поля превращается в обычную функцию один раз, поэтому оптимизатор вызывает её без повторной компиляции (листинг 3.6). Это существенно для Brython: компиляция строки при каждом из сотен тысяч вызовов градиентного спуска заняла бы минуты.

Листинг 3.6 – Компиляция введённой функции

```
def compile_function(expression, dimension):
    source = expression.strip().replace("^", "**")
    names = VARIABLES[:dimension]
    for name in _identifiers(source):
        if name not in _SAFE and name not in names:
            raise ValueError("unknown name: " + name)
    scope = dict(_SAFE)
    scope["__builtins__"] = {}
    func = eval("lambda " + ", ".join(names) + ": (" + source + ")", scope)

    def f(point):
        return float(func(*point))
    return f
```

Для функции двух переменных страница строит тепловую карту (значения берутся по логарифмической шкале) и рисует траекторию поиска, для функции одной переменной — график с точками траектории.

3.6 Окно и сервер

Класс `SiteServer` (модуль `lab3/server.py`) — встроенный `http.server` с корнем в каталоге `lab3/`: страницы лежат в `/site/`, общий модуль — в корне. Окно `python main.py --lab 3 --gui` показывает сайт в элементе WebView, кнопки открывают вкладки, справа находится список проверок.

4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

На рисунке 4.1 показано окно: слева сайт со вкладкой корней многочлена $x^3 - 6x^2 + 11x - 6$, справа — шесть сценариев, все выполнены успешно.

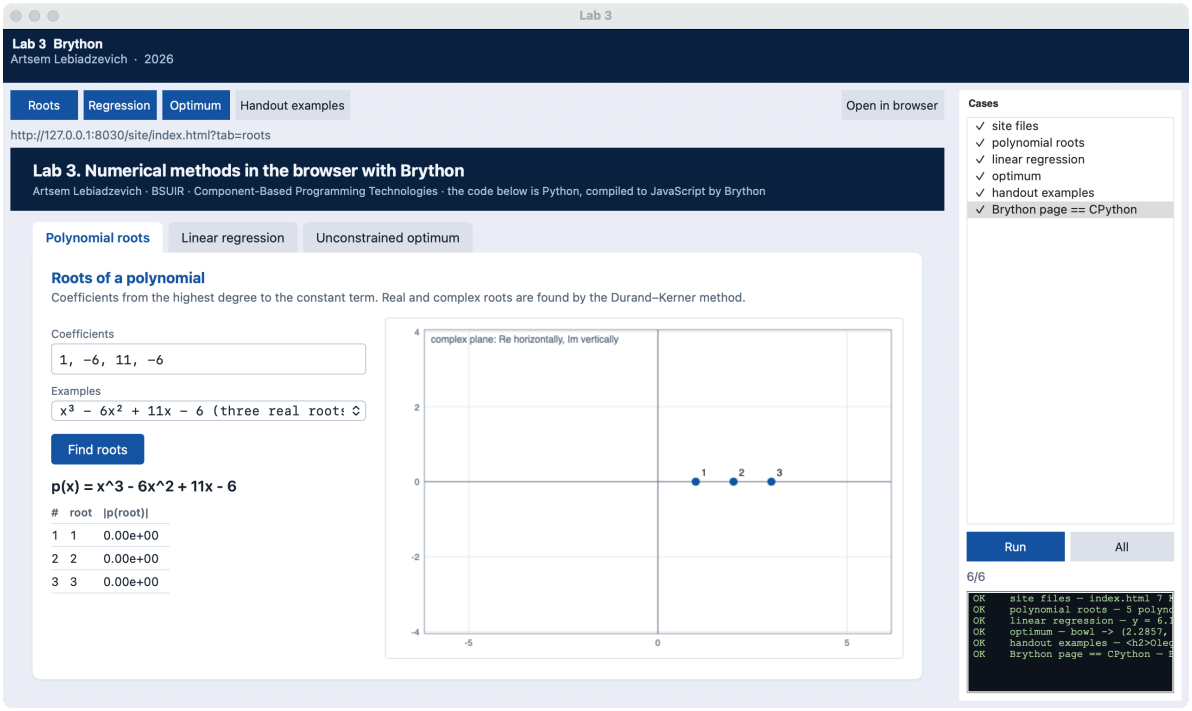


Рисунок 4.1 – Окно лабораторной работы: сайт Brython и список проверок

Вкладка регрессии (рисунок 4.2) показывает десять наблюдений, прямую $y = 6{,}139x + 95{,}33$ ($R^2 = 0{,}995$) и прогноз на четыре шага вперёд и в точке $x = 20$; оранжевая полоса — приближённый 95 % интервал, он расширяется по мере удаления от данных.

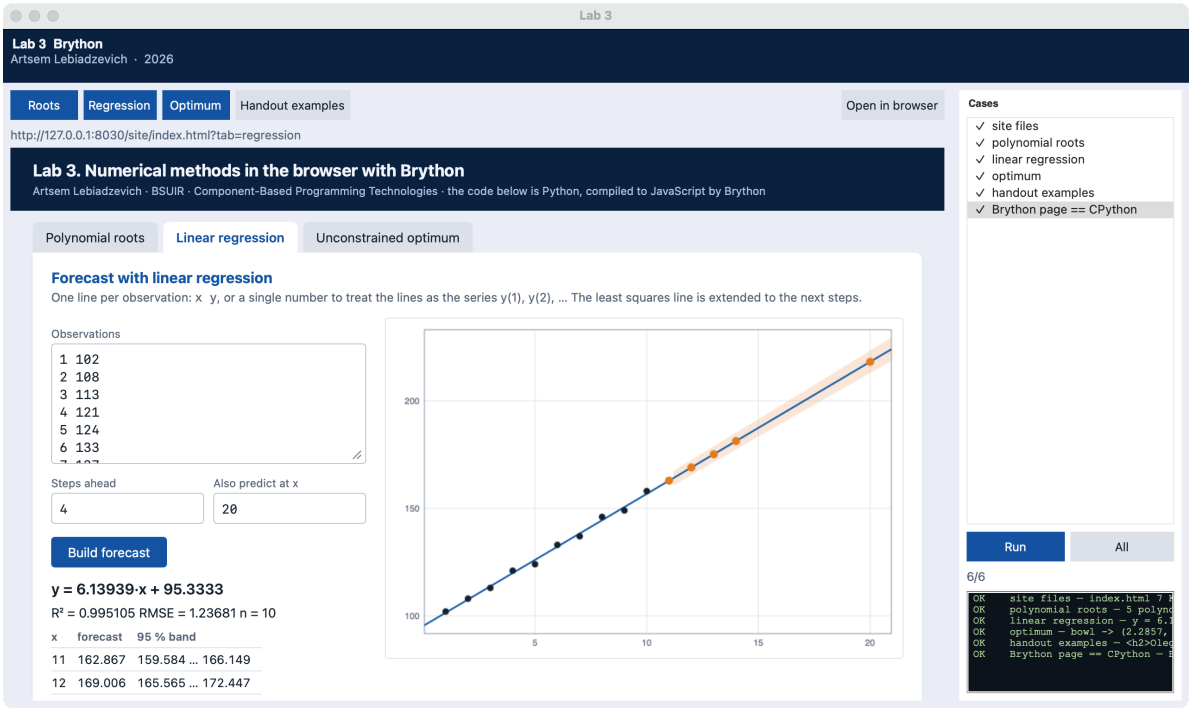


Рисунок 4.2 – Прогноз линейной регрессии

На рисунке 4.3 найден минимум функции $(x-1)^2+2(y+2)^2+xy$ методом Нелдера — Мида. Аналитическое решение системы $2(x-1)+y=0$, $4(y+2)+x=0$ даёт точку $(16/7,\ -18/7)\approx(2{,}2857;\ -2{,}5714)$, она совпадает с найденной за 65 итераций.

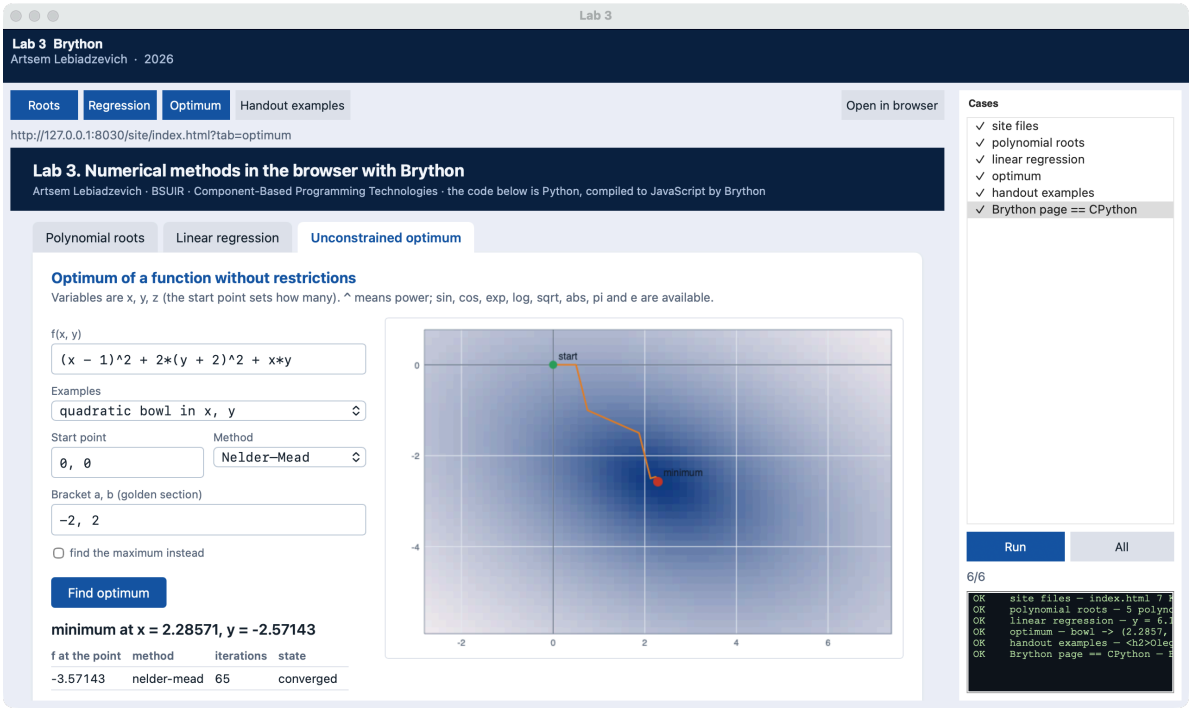


Рисунок 4.3 – Минимум функции двух переменных: тепловая карта и траектория поиска

Примеры из методички (рисунок 4.4) выполняются без изменений,

кроме явного `import browser.html`.

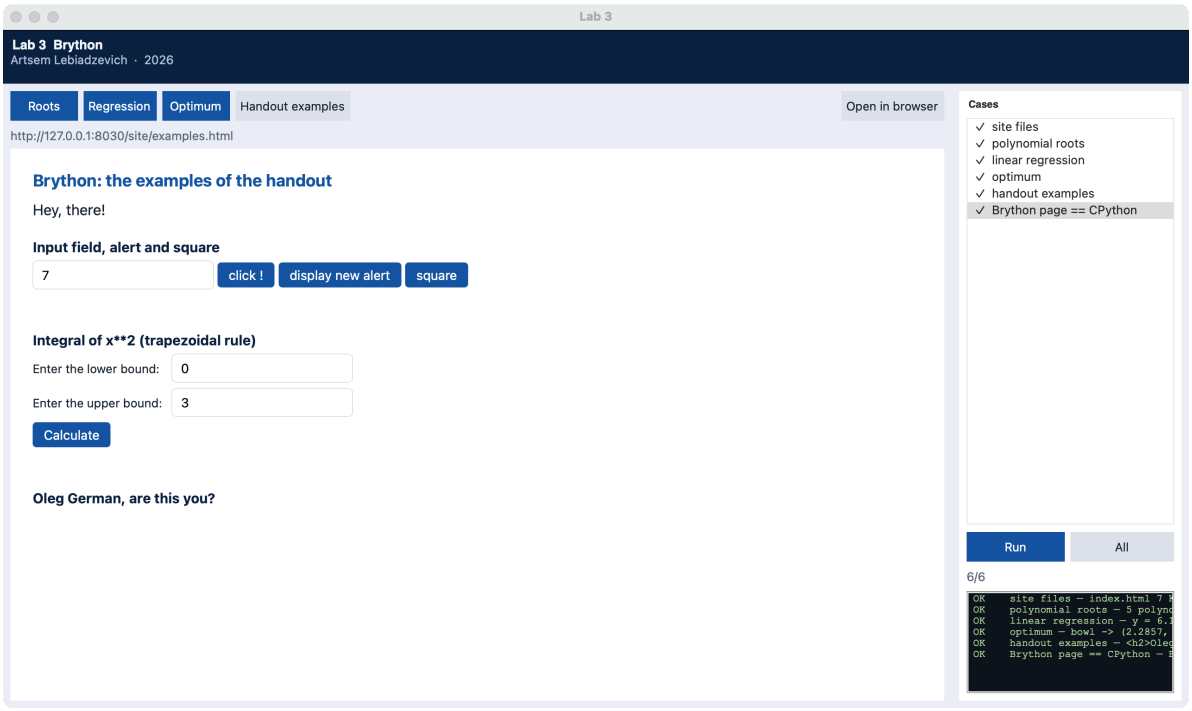


Рисунок 4.4 – Примеры из методических материалов

Команда `python main.py --lab 3` выполняет те же сценарии без окна. Её вывод приведён в листинге 4.1. Последняя строка — главная проверка: браузер без окна открывает страницу, Brython выполняет `app.py`, страница записывает вычисленные значения в DOM, и тест сравнивает 26 значений с результатами CPython.

Листинг 4.1 – Вывод программы

```
x^3 - 6x^2 + 11x - 6 -> 1, 2, 3
x^2 + 1 -> -i, i
x^4 - 1 -> -1, -i, i, 1
site_files -> index.html 7 KiB, app.py 14 KiB, brython.js 1430 KiB, numerics.py 18 KiB
polynomial_roots -> 5 polynomials, max |p(root)| = 8.9e-16; x^2+1 -> ±i
regression -> y = 6.1394x + 95.333, R² = 0.9951, x=11 -> 162.87
optimum -> bowl -> (2.2857, -2.5714), Rosenbrock -> (1.0000, 1.0000) in 126 steps, max of sin·exp at x = 1.4711
handout_examples -> <h2>Oleg German, are this you?</h2> was created by browser.html.H2 and is in the DOM
brython_page -> Brython 3.15.0 in headless browser: 26 values equal CPython's
```

Сводка по результатам приведена в таблице 4.1.

Таблица 4.1 – Проверка численных методов

Задача	Исходные данные	Результат
корни	$x^3 - 6x^2 + 11x - 6$	1, 2, 3
корни	$x^2 + 1$	$-i$, i
корни	$x^4 - 1$	-1, $-i$, i, 1
корни	$2x^4 - 8x^2$	-2, 0, 0, 2
регрессия	10 точек	$y = 6{,}139x + 95{,}33$, $R^2 = 0{,}995$; в точке 11 — 162,87
оптимум	квадратичная функция	$(2{,}2857;\ -2{,}5714)$, как в аналитическом решении
оптимум	функция Розенброка	Нелдер — Мид: $(1;\ 1)$ за 126 шагов; градиентный спуск достигает лимита 20 000 итераций
оптимум	$x^4 - 3x^2 + x$	локальный минимум $x \approx 1{,}131$ (градиентный спуск и золотое сечение)
максимум	$\sin x\, e^{-0{,}1x}$	$x = \operatorname{arctg} 10 \approx 1{,}4711$

Кроме сценариев окна написаны автоматические тесты: корни, регрессия (сверяется с `statistics.linear_regression`), методы оптимизации, отказ от посторонних имён в функции, раздача общего модуля сервером и сравнение результатов Brython и CPython. Все тесты проходят.

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Как создать клиентский сайт с помощью Brython? Подключить `brython.js` и `brython_stdlib.js`, вызвать `brython()` из `<body onload>`, написать код в блоке `<script type="text/python»` (внутри страницы или в файле `src="app.py"`). Элементы получают как `document["id"]`, обработчики подключают методом `bind("click f)`, результат выводят в `.text` или добавляют элементы оператором `document <= html.TAG(...)`.

2. Как подключить библиотеки Python в сценарии Brython? Стандартные модули подключаются обычным `import`: они берутся из `brython_stdlib.js`. Собственные модули — это файлы `.py` на сервере; Brython загружает их при импорте, а каталог поиска задаётся параметром `pythonpath` (`brython({pythonpath: ['/']})`). Скомпилированные пакеты, например numpy, недоступны, поэтому вычисления написаны на чистом Python.

3. Объясните решение лабораторной работы. Численные методы собраны в модуле `numerics.py`; `app.py` читает поля страницы, вызывает методы, строит таблицы и рисует на холсте; `index.html` содержит разметку. Сервер отдаёт браузеру тот же `numerics.py`, который используют тесты, а отдельный тест проверяет, что числа, вычисленные в браузере, совпадают с вычисленными в CPython.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучен фреймворк Brython и создан клиентский сайт, который находит корни многочлена, строит прогноз линейной регрессии и ищет оптимум функции без ограничений тремя методами. Весь код страницы, включая обработку событий, построение таблиц и рисование на холсте, написан на Python.

Главный практический вывод: Brython позволяет использовать один и тот же модуль в браузере и в обычном Python, поэтому численный код можно проверить автоматическими тестами на сервере, а затем выполнять у клиента без изменений. Ограничения подхода тоже видны: недоступны скомпилированные библиотеки, при загрузке страницы компилируется стандартная библиотека объёмом 5 МБ, а поведение зависит от версии фреймворка — примеры методички для версии 3.9 потребовали правки для версии 3.15.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Герман, О. В. Использование сценариев Python в HTML-документах с Brython: лекция и лабораторная работа по дисциплине «Технологии компонентного программирования» / О. В. Герман. — Минск: БГУИР, 2024.
2. Brython documentation [Электронный ресурс]. — Режим доступа: https://brython.info/static_doc/en/intro.html.
3. Press, W. H. Numerical Recipes: The Art of Scientific Computing / W. H. Press [et al.]. — 3rd ed. — Cambridge: Cambridge University Press, 2007. — 1256 p.
4. Nelder, J. A. A simplex method for function minimization / J. A. Nelder, R. Mead // The Computer Journal. — 1965. — Vol. 7, No. 4. — P. 308–313.